



ÉCOLE NATIONALE SUPÉRIEURE POLYTECHNIQUE DE YAOUNDÉ

Département Génie Informatique — 3^e Année

Introduction aux Réseaux — Semestre 2, 2025-2026

Business Core as a Service

BCaaS

*"Une infrastructure générique de gestion des métiers
inspirée de l'ingénierie des protocoles réseaux"*

Résumé

Ce rapport présente la conception et le développement de BCAAS (*Business Core as a Service*), une plateforme générique multi-tenant dédiée à la gestion standardisée des processus métier. Le projet s'inscrit dans le cadre du cours d'Introduction aux Réseaux de la troisième année de Génie Informatique.

L'idée centrale est de modéliser un système d'information métier en s'appuyant rigoureusement sur les concepts de l'ingénierie des protocoles réseaux : pile de couches, encapsulation, routage, adressage, qualité de service et plans de contrôle et de données. Cette analogie formelle permet de concevoir une architecture cohérente, modulaire et extensible.

La solution implémentée repose sur Spring Boot 3.5 / Java 21, PostgreSQL avec Liquibase, Spring Security avec JWT, et SpringDoc OpenAPI pour la documentation automatique de l'API. Le frontend est développé avec Next.js sous forme de Progressive Web App (PWA).

Mots-clés : Business Core, multi-tenant, protocoles réseaux, OSI, encapsulation, Spring Boot, API REST, Next.js.

Table des matières

Résumé	1
1 Introduction générale	6
1.1 Contexte du projet	6
1.2 Problématique	6
1.3 Objectifs	7
1.4 Structure du document	7
2 Analogie formelle entre réseaux et métier	8
2.1 Table de correspondance réseau-métier	9
2.2 Flux de traitement d'une requête	10
2.3 Modélisation du multi-tenant comme réseau virtuel	10
2.4 Plan de contrôle et plan de données	10
3 Architecture globale de la solution	12
3.1 Vue d'ensemble	12
3.2 Description des couches	12
3.2.1 Couche 1 — Infrastructure	12
3.2.2 Couche 2 — Transport & Messaging	13
3.2.3 Couche 3 — Tenant & Routing	13
3.2.4 Couche 4 — Context & Policy	13
3.2.5 Couche 5 — Business Capabilities	13
3.3 Architecture micro services	13
4 Conception détaillée	15
4.1 Modèle de données générique	15
4.2 Diagramme d'analyse	17
4.2.1 Diagramme de contexte	17
4.2.2 Diagramme de packages	17
4.2.3 Diagramme de classes métier	19

5	Implémentation technique	20
5.1	Pile technologique	20
5.2	Structure du projet Spring Boot	20
6	Protocole d'échange pour un cas métier concret	22
6.1	Cas choisi : Création d'un Business et ajout au catalogue de services . .	22
6.1.1	Protocole formel d'échange	22
7	Stratégie multi-tenant et isolation	25
7.1	Stratégie actuelle : colonne <code>tenant_id</code>	25
7.2	Stratégie future : isolation par schéma PostgreSQL	25
7.3	Provisionnement d'un nouveau tenant	26
8	Observabilité et QoS métier	27
8.1	Proposition d'observabilité	27
8.1.1	Logs structurés	27
8.1.2	Métriques (inspirées des KPIs réseau)	27
8.1.3	Tracing distribué	27
8.2	QoS métier : niveaux de service différenciés	28
9	Frontend Next.js	29
9.1	Architecture du frontend	29
9.2	Choix techniques	29
9.3	Pages principales	30
9.3.1	Site vitrine	30
9.3.2	Tableau de bord admin	30
10	Conclusion et perspectives	31
10.1	Bilan	31
10.2	Perspectives	31
10.3	Apports personnels	32

Table des figures

2.1	Flux bout-en-bout d'une requête dans BCAAS — analogie OSI	10
3.1	Pile protocolaire métier de BCAAS et correspondance avec le modèle OSI	12
3.2	Architecture de BCAAS	14
4.1	Diagramme de packages de l'architecture du système	18

Liste des tableaux

2.1	Correspondance formelle entre concepts réseaux et concepts métier . . .	9
4.1	Entités du modèle de données – Business Core	16
5.1	Pile technologique de BCAAS	20
7.1	Comparaison des stratégies d’isolation	26
8.1	Métriques proposées pour BCAAS	28
8.2	Niveaux de QoS proposés	28
9.1	Technologies frontend	29

Chapitre 1

Introduction générale

1.1 Contexte du projet

Dans le domaine du génie logiciel, chaque nouveau projet métier — qu’il s’agisse d’une pharmacie, d’une agence immobilière, d’un cabinet médical ou d’une école — reconstruit invariablement les mêmes fondations : gestion des acteurs, des ressources, des règles, des workflows, des notifications et de la traçabilité. Cette duplication systématique représente un gaspillage considérable en temps de développement, en maintenance et en cohérence architecturale.

Parallèlement, l’ingénierie des réseaux a résolu depuis des décennies un problème structurellement similaire : faire communiquer de façon fiable, sécurisée et interopérable des entités hétérogènes sur une infrastructure partagée. Le modèle OSI et la suite TCP/IP proposent une décomposition en couches aux responsabilités clairement délimitées, un mécanisme d’encapsulation des données, un adressage précis des destinataires, une qualité de service différenciée et une séparation nette entre plan de contrôle et plan de données.

1.2 Problématique

Question directrice

Comment concevoir un noyau métier générique, réutilisable et configurable qui permette à n’importe quelle instance métier de fonctionner sans reconstruire les fondations, en s’appuyant formellement sur les concepts de l’ingénierie des protocoles réseaux ?

1.3 Objectifs

Les objectifs du projet sont les suivants :

1. Établir une analogie formelle et rigoureuse entre les concepts réseaux (couches OSI, encapsulation, routage, QoS, multi-tenant) et les composants d'une architecture métier générique.
2. Concevoir une architecture modulaire multi-tenant en cinq couches protocolaires.
3. Implémenter un API de provisionnement de tenants avec modèle de données générique configurable.
4. Documenter et exposer l'API via Swagger UI versionné.
5. Développer un frontend Next.js comprenant un site vitrine et un tableau de bord d'administration.

1.4 Structure du document

Ce rapport est organisé en plusieurs chapitres. Le chapitre 2 formalise l'analogie entre réseaux et métier. Le chapitre 3 présente l'architecture globale de la solution. Le chapitre 4 détaille la conception des composants. Le chapitre 5 décrit l'implémentation technique. Le chapitre 9 présente le frontend. Enfin, le chapitre 10 conclut et ouvre sur des perspectives d'évolution.

Chapitre 2

Analogie formelle entre réseaux et métier

2.1 Table de correspondance réseau-métier

TABLE 2.1 – Correspondance formelle entre concepts réseaux et concepts métier

Concept réseau	Définition réseau	Équivalent métier dans BCaaS
Adresse IP	Identifiant unique d'un nœud	<code>tenant_id</code> + <code>domain</code> + <code>service</code>
En-tête de paquet	Métadonnées d'acheminement	<code>tenant_id</code> , <code>actor_id</code> , <code>trace_id</code> , SLA
Payload / SDU	Données applicatives	Corps de la requête métier
Encapsulation	Ajout d'en-têtes à chaque couche	Enrichissement du contexte au passage des couches
Routage	Sélection du chemin vers la destination	Résolution <code>tenant</code> → <code>service</code> → <code>instance</code>
NAT / Proxy	Traduction d'adresses	API Gateway : réécriture et normalisation
VLAN	Isolation logique sur réseau partagé	Schéma isolé par <code>tenant</code>
TCP (fiable)	Transport avec accusé de réception	REST synchrone avec <code>retry</code> et <code>idempotence</code>
QoS	Priorité et garanties de bande passante	SLA par <code>tenant</code> , priorité de transaction
TTL (Time To Live)	Durée de vie d'un paquet	Expiration de token JWT
Control plane	Gestion des routes (BGP, OSPF)	Admin : configuration <code>tenants</code> , règles, SLA
Data plane	Acheminement des paquets	Exécution des transactions métier
Firewall	Filtrage de paquets	Spring Security, contrôle d'accès API

2.2 Flux de traitement d'une requête

Le traitement d'une requête dans BCAAS suit exactement le modèle OSI descendant (émetteur) puis ascendant (récepteur) :

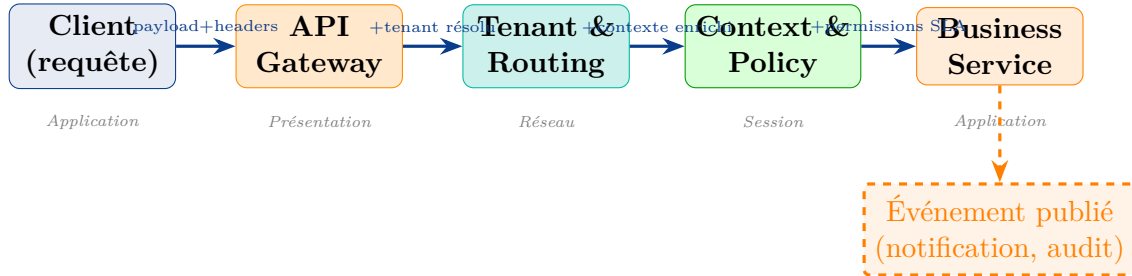


FIGURE 2.1 – Flux bout-en-bout d'une requête dans BCAAS — analogie OSI

2.3 Modélisation du multi-tenant comme réseau virtuel

Analogie : VLAN ↔ Tenant

Dans un réseau, un VLAN isole logiquement plusieurs groupes de machines sur la même infrastructure physique. Chaque trame VLAN porte un tag (802.1Q) qui identifie son appartenance. Dans BCAAS, chaque requête porte un `tenant_id` qui isole les données, les règles et les workflows d'un client sur l'infrastructure partagée.

VLAN tag $\equiv$ `tenant_id`, Réseau physique $\equiv$ Infrastructure partagée

2.4 Plan de contrôle et plan de données

Analogie : SDN ↔ BCaaS Admin

Dans les réseaux définis par logiciel (SDN), le control plane configure les règles de routage de façon centralisée, tandis que le data plane achemine les paquets. Cette séparation permet de modifier le comportement du réseau sans interruption de service.

Dans BCAAS, l'interface d'administration (control plane) configure tenants, règles métier, SLA et workflows. Le moteur d'exécution (data plane) traite les transactions en temps réel en appliquant ces configurations.



Chapitre 3

Architecture globale de la solution

3.1 Vue d'ensemble

L'architecture de BCAAS est organisée autour de cinq couches protocolaires, directement inspirées du modèle OSI, et d'une séparation claire entre control plane et data plane.

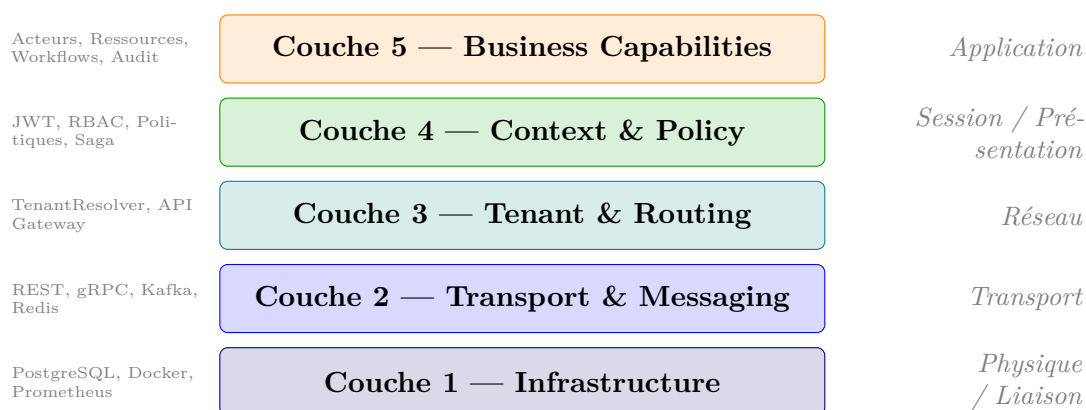


FIGURE 3.1 – Pile protocolaire métier de BCAAS et correspondance avec le modèle OSI

3.2 Description des couches

3.2.1 Couche 1 — Infrastructure

Cette couche constitue le substrat technique. Elle regroupe la base de données PostgreSQL 16 avec ses migrations Liquibase, le cache Redis et le bus de messages Apache Kafka. Elle correspond aux couches physique et liaison du modèle OSI : elle transporte les données sans connaître leur signification.

3.2.2 Couche 2 — Transport & Messaging

Analogie directe avec la couche transport OSI (TCP/UDP). Elle gère le mode de transport des échanges (synchrone via REST, ou asynchrone via Kafka), les stratégies de retry, les timeouts et l'idempotence des opérations. Le choix TCP $\equiv$ REST (fiable, ordonnée) ou UDP $\equiv$ Kafka (rapide, meilleur effort) est explicite dans la conception.

3.2.3 Couche 3 — Tenant & Routing

Analogie avec la couche réseau. Le *Tenant Resolver* joue le rôle d'un routeur : il détermine, à partir de l'en-tête de la requête (`tenant_id` ou sous-domaine), quelle instance de configuration utiliser, vers quel service router la requête. C'est ici que réside la logique de découverte de service.

3.2.4 Couche 4 — Context & Policy

Analogie avec les couches session et présentation. Cette couche enrichit le contexte d'exécution : elle valide le token JWT, décode les rôles et permissions, applique les politiques de sécurité propres au tenant, détermine le niveau de SLA et initialise le contexte de saga pour les transactions longues. L'en-tête métier (H_C) est construit ici.

3.2.5 Couche 5 — Business Capabilities

Couche applicative. Elle contient les services métier génériques : gestion des acteurs, des ressources, des règles, des workflows, des documents et de l'audit. Ces services opèrent sur des modèles de données configurables par tenant, sans connaissance des couches inférieures.

3.3 Architecture micro services

Cette architecture conserve la séparation logique en services cohérents tout en simplifiant le déploiement et la coordination initiale.

Les modules métier sont organisés en packages cohérents (actors, businesses, service-catalogues, invoices, analytics) et communiquent via des appels directs en mémoire. Cela permet de respecter l'architecture hexagonale et le Domain Driven Design sans la complexité d'un mesh de microservices.

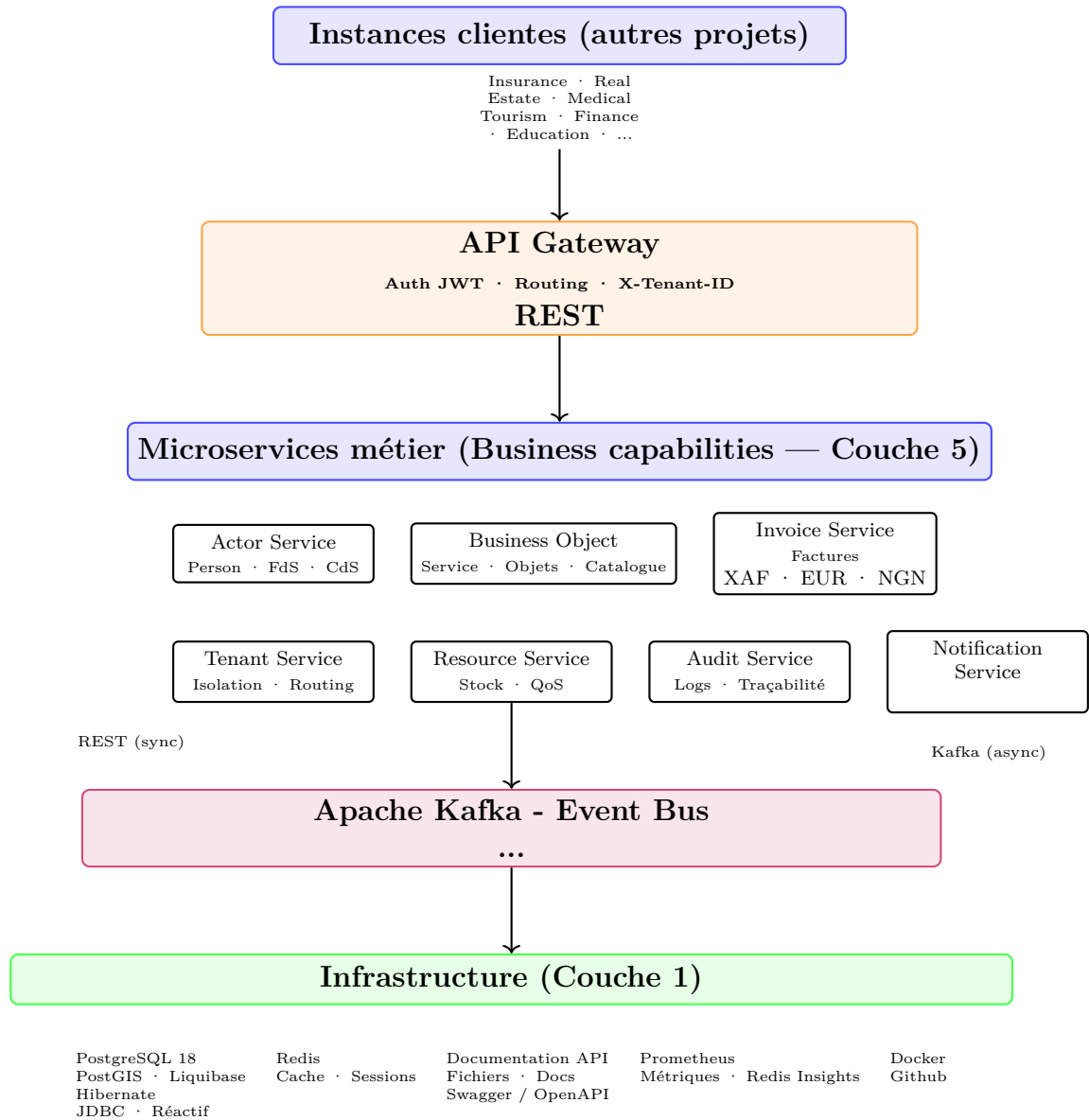


FIGURE 3.2 – Architecture de BCAAS

Chapitre 4

Conception détaillée

4.1 Modèle de données générique

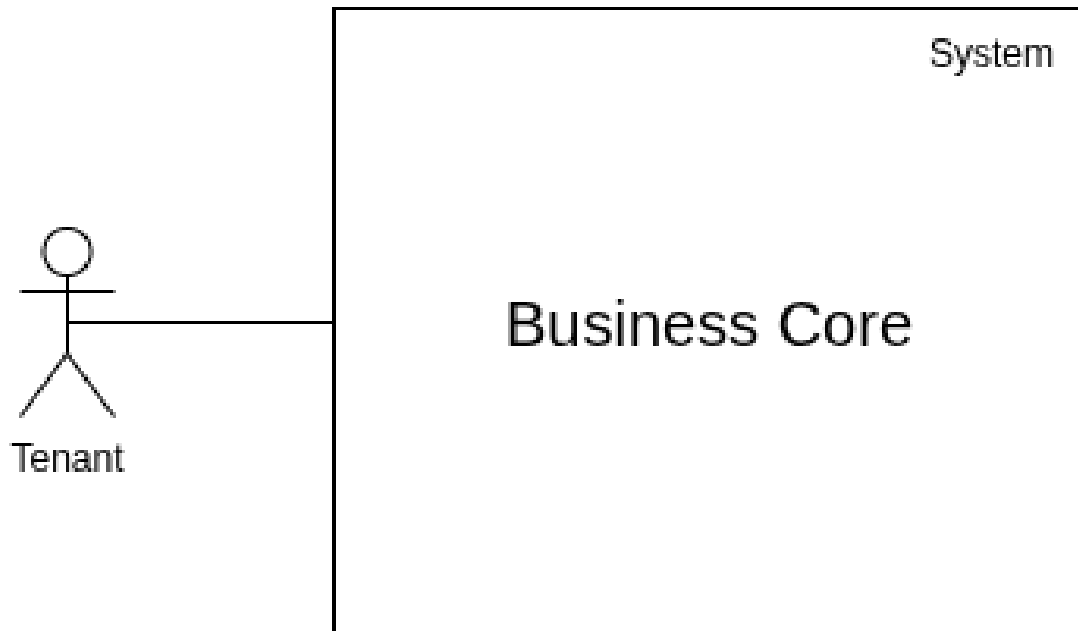
Le défi central de BCAAS est de modéliser des entités métier hétérogènes (médecin, livre, appartement, produit agricole) avec un schéma de données unique. Nous adoptons le modèle entités relations associé au modèle UML.

TABLE 4.1 – Entités du modèle de données – Business Core

Table	Attributs
tenant	<u>id_Tenant</u> , Name, IsActive, CreatedAt
user	<u>Id_User</u> , FirstName, LastName, Email, Gender, Password, IsActive, Country, Phone, CreatedAt, <i>#id_Tenant</i>
actor	<u>Id_Actor</u> , NomProfessionnel, Speciality, bio, CreatedAt, Role, <i>#Id_User</i>
formation	<u>Id_Formation</u> , NomFormation, Description, Type_Formation, Niveau
media	<u>Id_Media</u> , Type, Url, Label, CreatedAt, <i>#Id_Business</i>
business	<u>Id_Business</u> , NameBusiness, Domain, Type_of_InvolvedActors, RequiredJobProfile, CreatedAt
business_rule	<u>Id_BusinessRule</u> , RuleKey, RuleValue, Description, CreatedAt
ressource	<u>Id_Ressource</u> , NameRes, Type, Quantity, Description, CreatedAt, Disponibility
service_catalogue	<u>Id_SerCatalogue</u> , NameSerCatalogue, Description, Currency, IsAvailable, CreatedAt
portfolio	<u>Id_Portfolio</u> , title, Description, Status, <i>#Id_Business</i>
service_request	<u>Id_ServiceRequest</u> , TraceId, ServiceName, Description, Status, <i>#Id_Actor</i> , <i>#Id_Portfolio</i>
message	<u>id_Message</u> , content, isRead, createdAt, <i>#Id_ServiceRequest</i>
notification	<u>id_Notification</u> , title, message, isRead, createdAt, <i>#Id_ServiceRequest</i>
invoice	<u>id_Invoice</u> , Amount, Currency, Status, UsedAt, PaidAt, IssueAt, <i>#Id_ServiceRequest</i>

4.2 Diagramme d'analyse

4.2.1 Diagramme de contexte



4.2.2 Diagramme de packages

Le diagramme de packages présenté en Figure 4.1 propose une organisation modulaire du système selon une architecture en couches ou en domaines fonctionnels. Chaque package représenté correspond à un sous-système ayant des responsabilités bien définies.

Tenant L'acteur **Tenant** suggère une architecture multi-tenant, où chaque client (organisation ou locataire) dispose de sa propre isolation des données et configurations. Ce package agit probablement comme un contexte global d'exécution permettant aux autres modules de fonctionner de manière isolée par tenant.

Authentification et gestion des acteurs Le package **AUTHENTIFICATION** gère les processus d'identification et de vérification des tenants. Il interagit directement avec **GESTION DES ACTEURS** qui centralise la définition, les rôles et les permissions des différentes entités utilisatrices (administrateurs, clients, fournisseurs, etc.). Cette séparation permet de dissocier le mécanisme technique d'authentification de la gestion administrative des profils.

Gestion des business et services Les packages **GESTION DES BUSINESS** et **GESTION DES SERVICES** couvrent le cœur métier. Le premier traite des règles propres à l'activité

de l'entreprise, tandis que le second coordonne l'information, l'orchestration ou la souscription aux services proposés. Leur découpage facilite l'évolution indépendante des règles métier et des mécanismes de service.

Gestion des médias, ressources et factures Le package **GESTION DES MEDIA** est dédié aux contenus multimédias (images, vidéos, documents). **GESTION DES RESSOURCES** supervise les actifs techniques ou matériels nécessaires au fonctionnement (serveurs, licences, matériel, équipements). Enfin, **GESTION DES FACTURES** assure la chaîne de facturation, de la génération des factures au suivi des paiements.

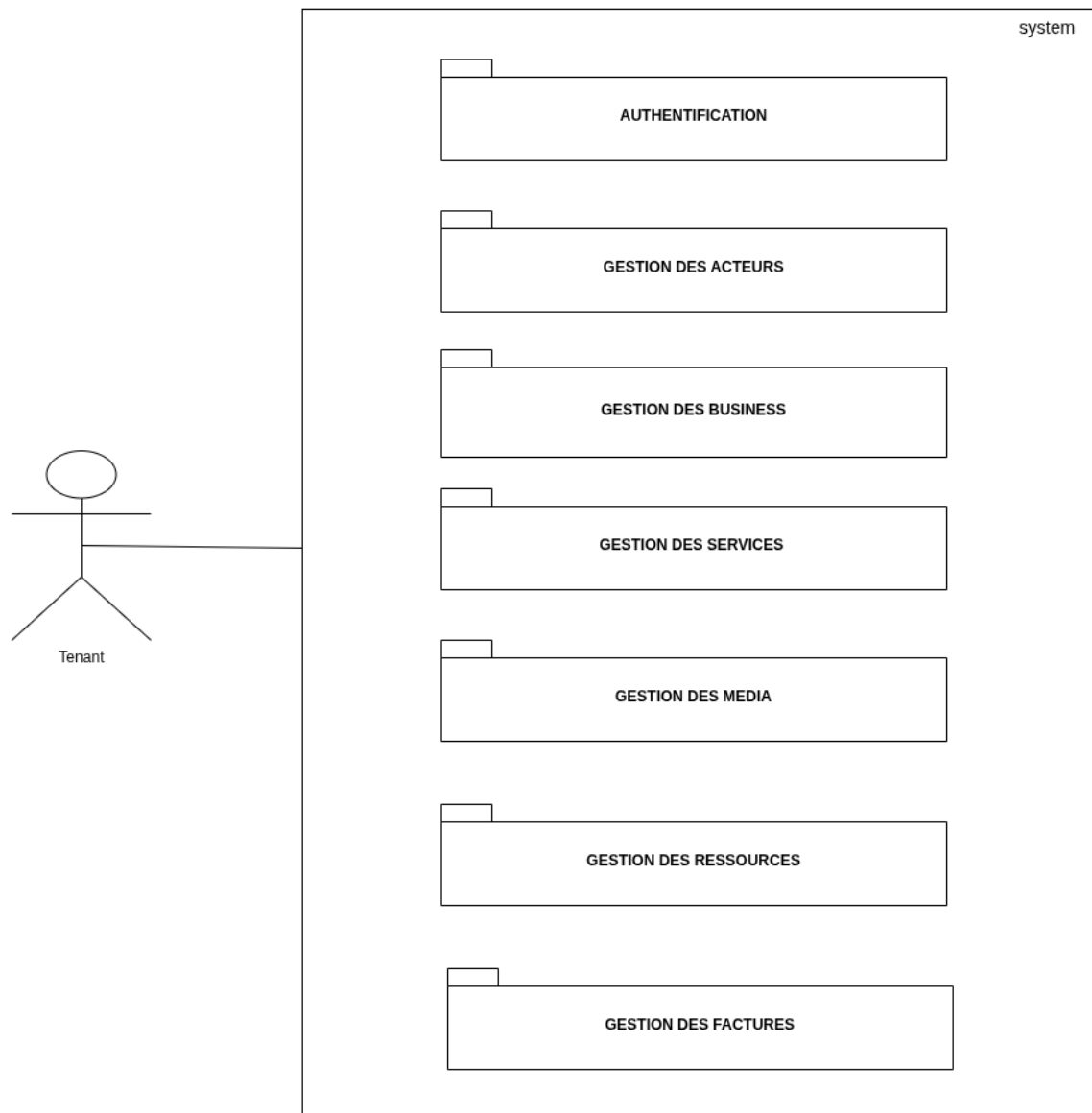
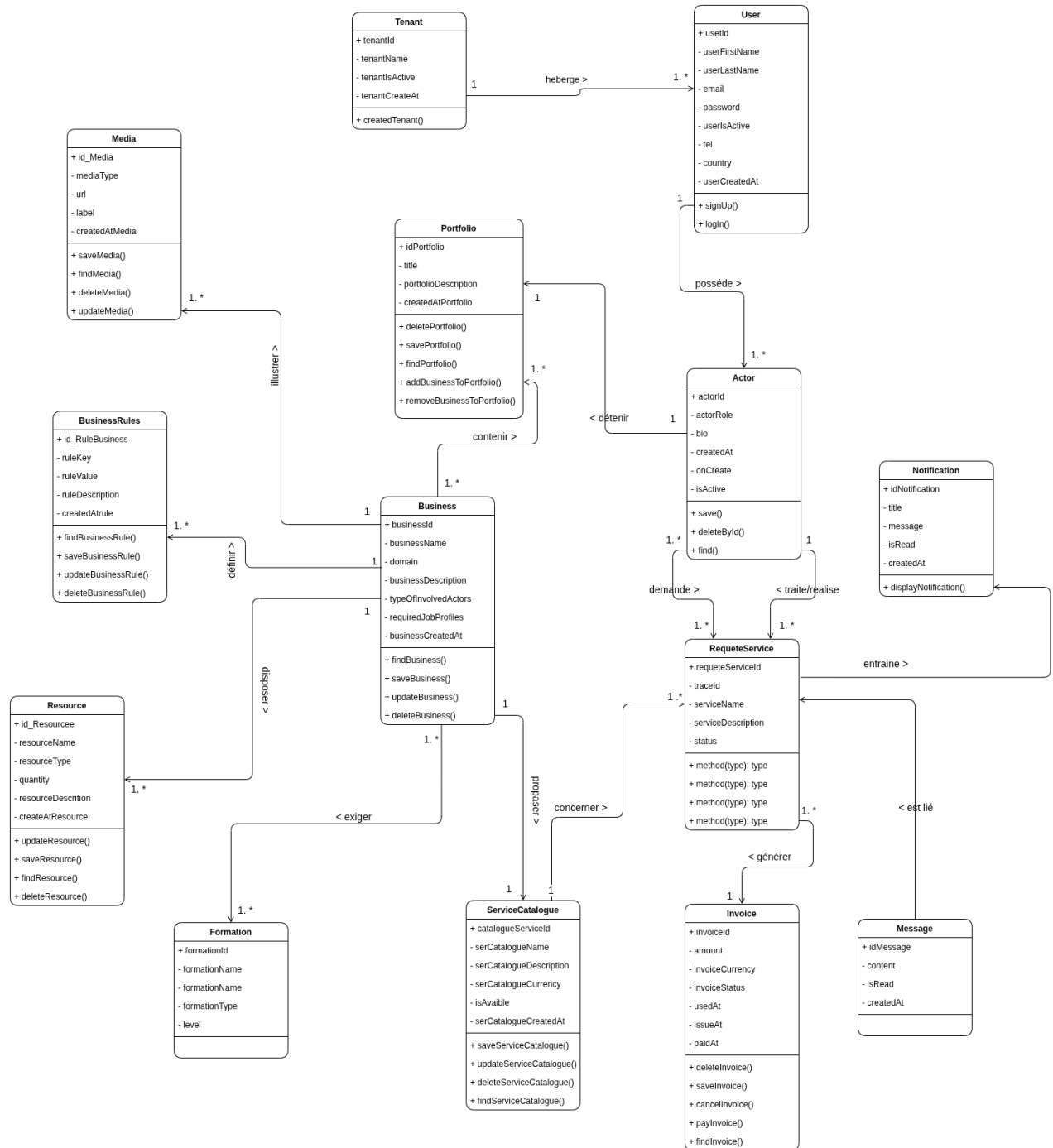


FIGURE 4.1 – Diagramme de packages de l'architecture du système

4.2.3 Diagramme de classes métier



Chapitre 5

Implémentation technique

5.1 Pile technologique

TABLE 5.1 – Pile technologique de BCAAS

Composant	Technologie	Version
Backend Framework	Spring Boot	3.5.x
Langage	Java	21 LTS
Base de données	PostgreSQL	16
Migrations	Liquibase	4.x
Sécurité	Spring Security + JWT	jjwt 0.12.6
Documentation API	SpringDoc OpenAPI	2.8.6
Build	Maven	3.9+
Tests	JUnit 5 + Mockito	≥16 tests
Conteneurisation	Docker + Docker Compose	latest
Frontend	Next.js 14	PWA
Mobile	React Native	Expo

5.2 Structure du projet Spring Boot

Le projet suit l'architecture hexagonale (*ports et adaptateurs*) combinée avec le Domain-Driven Design (DDD). Cette organisation garantit que le domaine métier reste indépendant des détails techniques.

```
1  backend/
2  +-- docker-compose.yml
3  +-- Dockerfile
4  +-- docs/
5  +-- pom.xml
6  +-- src/
7  +-- main/
```

```

8      +-- java/com/bizcore/bizcore_backend/
9          +-- auth/                                # Authentification JWT
10         +-- config/                              # Configurations (Kafka,
Redis, Security)
11         +-- controller/                          # Controllers REST (Couche 3)
12         +-- domain/                              # Entites JPA (Couche 5)
13         +-- dto/                                  # Objets de transfert
14         +-- exception/                           # Gestion des erreurs
15         +-- listener/                             # Listeners Kafka
16         +-- repository/                           # Repositories Spring Data
17         +-- security/                             # JWT, TenantFilter (Couche
4)
18         +-- service/                             # Services metier
19     +-- resources/
20         +-- application-dev.yml
21         +-- application-prod.yml
22         +-- application.properties
23         +-- db/changelog/                         # Migrations Liquibase
24 +-- test/                                         # Tests JUnit 5 + Mockito
25 +-- target/                                       # Dossier compile (ignorer)

```

Listing 5.1 – Structure du projet backend BCaaS

Chapitre 6

Protocole d'échange pour un cas métier concret

6.1 Cas choisi : Création d'un Business et ajout au catalogue de services

Ce cas métier illustre la séquence complète d'enregistrement d'une nouvelle activité professionnelle (ex : auto-école, cabinet de conseil) dans un tenant, puis l'ajout de services dans son catalogue. Ce processus correspond à deux requêtes distinctes mais liées.

6.1.1 Protocole formel d'échange

Protocole : Création d'un Business

Endpoint : POST /api/businesses

En-têtes obligatoires :

- X-Tenant-ID : identifiant du tenant (analogie : adresse réseau)
- Authorization: Bearer <JWT> : authentification (analogie : ticket de session)

Corps de la requête (payload) :

```
1  {
2      "name": "Auto-Ecole Centrale",
3      "domain": "driving_school",
4      "description": "Numero 1 dans la formation auto",
5      "typeOfInvolvedActors": ["instructor", "student"],
6      "requiredJobProfile": "moniteur"
7      "neededEducation": "Baccaleaureat",
```

```

8     "neededTraining": "Permis A, B, G"
9 }

```

Réponse (succès 201 Created) :

```

1 {
2     "id": "550e8400-e29b-41d4-a716-446655440000",
3     "nameBusiness": "Auto-Ecole Centrale",
4     "status": "ACTIVE",
5     "createdAt": "2025-05-04T10:00:00Z"
6 }

```

Codes d'erreur (analogie : ICMP) :

- 400 Bad Request : payload invalide
- 401 Unauthorized : JWT manquant ou invalide
- 403 Forbidden : rôle insuffisant
- 404 : tenant inexistant
- 409 Conflict : business déjà existant

Protocole : Ajout au catalogue de services

Endpoint : POST /api/service-catalogues

En-têtes : identiques au protocole précédent + corrélation optionnelle X-Trace-ID.

Corps de la requête :

```

1 {
2     "businessId": "550e8400-e29b-41d4-a716-446655440000",
3     "name": "Cours de conduite",
4     "description": "Formation theorique et pratique de 3
5         mois ",
6     "basePrice": 150000,
7     "currency": "XAF",
8     "isAvailable": true
9 }

```

Réponse (succès) :

```

1 {
2     "id": "660e8400-e29b-41d4-a716-446655441111",
3     "businessId": "550e8400-e29b-41d4-a716-446655440000",
4     "businessName": "Auto-ecole",
5     "name": "Cours de conduite ",
6     "description": "Formation theorique pratique de 3 mois ",
7     "basePrice": 150000,
8     "currency": "XAF",

```

```
9      "isAvailable": true
10      "createdAt": "2025-05-04T10:05:00Z"
11    }
```

Sémantique du transport :

- Mode : **synchrone** (REST) — analogie TCP
- Timeout : 30 secondes
- Idempotence : non (chaque POST crée une nouvelle ressource)
- Retry : recommandé avec backoff exponentiel (max 3 tentatives)

Chapitre 7

Stratégie multi-tenant et isolation

7.1 Stratégie actuelle : colonne `tenant_id`

Dans l'implémentation actuelle, tous les tenants partagent les mêmes tables PostgreSQL. L'isolation est assurée par la présence systématique d'une colonne `tenant_id` de type UUID dans chaque table métier.

```
1      -- Toutes les requetes incluent implicitement cette clause
2      SELECT * FROM businesses
3      WHERE tenant_id = current_tenant_id()
4      AND id = :businessId;
```

Listing 7.1 – Exemple de requête avec filtre tenant

Garantie d'isolation

Le `TenantFilter` injecte le `tenant_id` dans le contexte d'exécution (`ThreadLocal`). Tous les repositories sont configurés pour filtrer automatiquement sur cette valeur. Aucune requête ne peut accidentellement accéder aux données d'un autre tenant.

7.2 Stratégie future : isolation par schéma PostgreSQL

Pour les tenants ayant des exigences renforcées (données sensibles, conformité RGPD, volumétrie élevée), une isolation par schéma physique est prévue.

L'approche retenue pour la **phase 2** est hybride :

- Tenant standard → table unique avec `tenant_id`

TABLE 7.1 – Comparaison des stratégies d’isolation

Stratégie	Isolation	Performance	Complexité
Colonne <code>tenant_id</code>	Logique	Bonne (index)	Faible
Schéma séparé	Physique	Très bonne	Moyenne
Base de données séparée	Totale	Variable	Élevée

— Tenant premium → schéma dédié (`tenant_<id>`)

La résolution du schéma se fera dynamiquement via un `tenantResolver` qui modifie le search path de PostgreSQL :

```

1  -- Execute avant chaque requete du tenant premium
2  SET search_path TO tenant_abc123, public;
```

Listing 7.2 – Routage dynamique vers le schéma tenant

7.3 Provisionnement d’un nouveau tenant

Le provisionnement suit une séquence protocolaire en plusieurs étapes, analogue à l’attribution d’une adresse IP et à la configuration d’un VLAN.

1. Réception de la requête `POST /api/tenants`
2. Validation des informations (nom, domaine, SLA)
3. Génération d’un `tenant_id` unique (UUID v4)
4. Création des enregistrements dans `tenants`
5. Génération d’un token JWT initial
6. Retour de la configuration au client

Chapitre 8

Observabilité et QoS métier

8.1 Proposition d'observabilité

L'observabilité dans BCAAS suit les trois piliers classiques : logs, métriques, traces. Chaque pilier est mis en correspondance avec un concept réseau.

8.1.1 Logs structurés

```
1  {
2    "timestamp": "2025-05-04T10:05:23Z",
3    "level": "INFO",
4    "tenant_id": "550e8400-e29b-41d4-a716-446655440000",
5    "trace_id": "7e9a8b7c-6d5e-4f3a-2b1c-9d8e7f6a5b4c",
6    "actor_id": "actor_123",
7    "service": "BusinessService",
8    "operation": "createBusiness",
9    "duration_ms": 145,
10   "http_status": 201
11 }
```

Listing 8.1 – Exemple de log structuré (JSON)

8.1.2 Métriques (inspirées des KPIs réseau)

Le service `AnalyticsService` calcule déjà plusieurs indicateurs clés. Nous proposons de les exposer via Prometheus.

8.1.3 Tracing distribué

L'en-tête `X-Trace-ID` est propagé à travers tous les appels (analogie : numéro de séquence TCP ou identifiant de circuit virtuel). Il permet de reconstituer le chemin

TABLE 8.1 – Métriques proposées pour BCAAS

Métrique	Analogie réseau	Calcul
Taux de conversion	Taux de livraison de paquets	requêtes honorées / total
Latence moyenne	RTT (Round Trip Time)	Temps moyen de traitement
Débit (req/s)	Bande passante	Requêtes par minute
Taux d'erreur	Taux de perte	4xx/5xx / total
Panier moyen	Taille moyenne de payload	Chiffre d'affaires / nombre factures
Délai d'exécution	Délai de routage	Temps entre requête et fulfillment

complet d'une transaction.

Trace ID

Le `trace_id` est généré à l'entrée de l'API Gateway et transmis à chaque service interne, chaque appel à la base de données, chaque publication d'événement. Il joue le rôle du champ *Identifiant de flux* dans les protocoles réseau.

8.2 QoS métier : niveaux de service différenciés

Inspirée des classes de service DiffServ (Differentiated Services) dans les réseaux IP, BCAAS propose plusieurs niveaux de QoS pour les tenants.

TABLE 8.2 – Niveaux de QoS proposés

Niveau	Latence max	Débit garanti	Priorité sup-port	Isolation
BRONZE	5 secondes	10 req/s	Standard	Colonne tenant_id
SILVER	2 secondes	50 req/s	Prioritaire	Colonne + cache Redis
GOLD	500 ms	200 req/s	24/7 dédié	Schéma dédié
PLATINUM	100 ms	Illimité	Garantie contractuelle	Base dédiée

Chapitre 9

Frontend Next.js

9.1 Architecture du frontend

Le frontend de BCAAS est divisé en deux parties distinctes correspondant aux deux plans de l'architecture :

Site vitrine (data plane view) Présentation publique de l'API, documentation interactive, exemples d'utilisation et catalogue des instances métier disponibles.

Tableau de bord Admin (control plane) Interface sécurisée pour la gestion des tenants, le suivi des appels API, les statistiques d'utilisation et la configuration des règles.

9.2 Choix techniques

TABLE 9.1 – Technologies frontend

Besoin	Technologie	Justification
Framework	Next.js 14 (App Router)	SSR/SSG, SEO, PWA native
Styling	Tailwind CSS	Rapidité, cohérence design
UI Components	shadcn/ui	Accessibilité, personnalisable
Graphiques admin	Recharts	Léger, intégrable React
Auth	NextAuth.js	OAuth2 / JWT intégré
État global	Zustand	Simple, performant

9.3 Pages principales

9.3.1 Site vitrine

- / — Hero section avec présentation de l'approche protocolaire
- /architecture — Diagramme interactif des couches
- /docs — Documentation API (intégration Swagger UI)
- /instances — Galerie des instances métier disponibles
- /guides — Guide de provisionnement tenant

9.3.2 Tableau de bord admin

- /admin/dashboard — Vue globale : tenants actifs, appels/min, erreurs
- /admin/tenants — Gestion CRUD des tenants
- /admin/tenants/[id] — Détail : configuration, acteurs, ressources, règles
- /admin/api-calls — Journal des appels API avec filtres
- /admin/stats — Graphiques : volume, latence, répartition par tenant

Chapitre 10

Conclusion et perspectives

10.1 Bilan

Ce projet a démontré qu'une analogie rigoureuse entre l'ingénierie des protocoles réseaux et la conception d'une plateforme métier n'est pas une simple métaphore pédagogique : elle constitue une méthode de conception puissante. Les concepts de couches, d'encapsulation, de routage, d'adressage, de QoS et de séparation control/data plane ont guidé chaque décision architecturale de BCAAS.

Les réalisations principales sont :

- Une pile protocolaire métier en cinq couches formellement liée au modèle OSI.
- Un modèle de données générique et configurable par tenant implémenté en PostgreSQL.
- Une API REST documentée et versionnée avec Spring Boot 3.5 / Java 21.
- Une authentification JWT multi-tenant avec contrôle d'accès basé sur les rôles.
- Un protocole d'échange formalisé pour le cas métier "Business + Catalogue".
- Une stratégie d'isolation multi-tenant (colonne `tenant_id` évolutive vers schémas dédiés).
- Une proposition d'observabilité (logs, métriques KPIs) et de QoS différenciée.
- Un frontend Next.js avec site vitrine et tableau de bord d'administration.

10.2 Perspectives

Court terme Intégration de Redis pour le cache des configurations tenant et Kafka pour la propagation d'événements asynchrones.

Moyen terme Migration de l'architecture modulaire monolithique vers des micro-services déployés individuellement.

Long terme Marketplace d’instances métier permettant à des entreprises camerounaises et africaines de déployer leur propre configuration BCAAS sans développement.

10.3 Apports personnels

Ce projet a permis d’acquérir des compétences concrètes en architecture logicielle distribuée, en conception d’API REST professionnelle, en sécurité applicative (JWT, RBAC) et en modélisation de systèmes génériques. La démarche de pensée protocolaire — définir les contrats avant le code, modéliser les échanges comme des messages — constitue un réflexe d’ingénieur directement applicable à tout projet de systèmes d’information.

Bibliographie

- [1] Prof. Thomas Djotio Ndié *Cours Introduction aux Réseaux Locaux 2025 - 2026.pdf*, thomas.djotio@yowwyob.com, +237 675 518 880.
- [2] SpringDoc OpenAPI Documentation, <https://springdoc.org>, consulté en 2025.
- [3] ISO/IEC 7498-1, *Information technology — Open Systems Interconnection — Basic Reference Model*, ISO, 1994.
- [4] <https://elearningenspy.com/course/view.php?id=56>
- [5] <https://www.geeksforgeeks.org/system-design/single-tenant-vs-multi-tenant-architecture/>